

Good Coding Practices

Image Processing & Analysis for Life Scientists

Olivier Burri, Romain Guiet & Arne Seitz

```
// This is a comment on a line  
// This is another line of comment
```

```
/*  
 * This allows to make comments  
 * on several lines  
 * and is useful to separate  
 * "parts" of your code  
 *  
 */
```

- Comments make the code easier to understand for others but also to **YOU** when you'll re-use your own code.
- Comments should help understand the flow and logic of the code what you want to achieve and how you do it. It should not describe what you did.

1. Write down the important Steps of the code using `comments` as a starting point and as **placeholders** to see what needs to be done.
(Opening Image, Getting Metadata, Processing, Saving the Output...)
2. Test each part of your code individually (if possible) making sure that the result of one part is independent from the others
3. Assemble the parts together in the designated **placeholders**

Using Variables & Functions

- Use `variables` to ease the fine tuning of parameters
- Use **meaningful** variable names (rather than short meaningless ones)
`pixelSize` instead of `p`
- Use **meaningful** function names as well
`segmentImage()` instead of `myFunction()`
- Define **all** variables at the start of your code

- **Save often**
- Read error messages to find the faulty line in your code.
- In ImageJ Macro Language, the most common errors are
 - Forgetting the semicolon ; at the end of the line
 - String concatenation errors , forgetting a plus + or quotation mark ``